

Flutter Material Widget Notes

Generated: 2026-06-04

Scope

- This document focuses on Flutter Material widgets commonly used to define UI.
- For every widget: short description, syntax, keyword properties, pseudo code, and example.
- Flutter evolves over time, so check official API docs for latest parameters.

=====

1) MaterialApp

Description: Root Material configuration for routes, theme, localization, and app-wide behavior.

Syntax: `MaterialApp(title: 'App', theme: ThemeData(), home: HomePage())`

Keyword Properties: title, theme, darkTheme, themeMode, home, routes, navigatorKey, debugShowCheckedModeBanner

Pseudo Code:

```
create MaterialApp
set theme and routes
set home screen
```

Example:

```
MaterialApp(
  title: 'Shop App',
  theme: ThemeData(useMaterial3: true),
  home: const HomeScreen(),
)
```

2) Scaffold

Description: Main page layout shell with app bar, body, floating action button, drawers, and bottom bars.

Syntax: `Scaffold(appBar: AppBar(...), body: Widget())`

Keyword Properties: appBar, body, drawer, endDrawer, floatingActionButton, bottomNavigationBar, bottomSheet, backgroundColor

Pseudo Code:

```
build page frame
attach top bar and body
add optional bottom and overlay actions
```

Example:

```
Scaffold(
  appBar: AppBar(title: const Text('Dashboard')),
  body: const Center(child: Text('Hello')),
  floatingActionButton: FloatingActionButton(onPressed: () {}),
)
```

3) AppBar

Description: Material top app bar for title, actions, navigation icon, and tabs.

Syntax: `AppBar(title: Text('Title'), actions: [IconButton(...)])`

Keyword Properties: title, leading, actions, bottom, centerTitle, elevation, backgroundColor

Pseudo Code:

```
create top bar
set title
add actions and optional tabs
```

Example:

```
AppBar(
  title: const Text('Profile'),
  actions: [
    IconButton(icon: const Icon(Icons.search), onPressed: () {}),
  ],
)
```

4) Drawer

Description: Side navigation panel that slides from screen edge.

Syntax: `Drawer(child: ListView(children: [...]))`

Keyword Properties: `child`, `elevation`, `backgroundColor`, `shape`, `width`

Pseudo Code:

```
open side panel
render menu items list
navigate on selection
```

Example:

```
Drawer(
  child: ListView(
    children: const [
      DrawerHeader(child: Text('Menu')),
      ListTile(title: Text('Home')),
    ],
  ),
)
```

5) `NavigationBar`

Description: Material 3 bottom navigation component for top-level destinations.

Syntax: `NavigationBar(selectedIndex: i, destinations: [...], onDestinationSelected: fn)`

Keyword Properties: `selectedIndex`, `destinations`, `onDestinationSelected`, `height`, `labelBehavior`

Pseudo Code:

```
keep selected index in state
render destinations
update index on tap
```

Example:

```
NavigationBar(
  selectedIndex: index,
  onDestinationSelected: (i) => setState(() => index = i),
  destinations: const [
    NavigationDestination(icon: Icon(Icons.home), label: 'Home'),
    NavigationDestination(icon: Icon(Icons.person), label: 'Profile'),
  ],
)
```

6) `BottomNavigationBar`

Description: Material 2 style bottom navigation with items and selected tab.

Syntax: `BottomNavigationBar(items: [...], currentIndex: i, onTap: fn)`

Keyword Properties: `items`, `currentIndex`, `onTap`, `type`, `selectedItemColor`, `unselectedItemColor`, `showSelectedLabels`

Pseudo Code:

```
render bottom items
highlight current index
switch screen on tap
```

Example:

```
BottomNavigationBar(
  currentIndex: current,
  onTap: (i) => setState(() => current = i),
  items: const [
    BottomNavigationBarItem(icon: Icon(Icons.home), label: 'Home'),
    BottomNavigationBarItem(icon: Icon(Icons.settings), label: 'Settings'),
  ],
)
```

7) `NavigationRail`

Description: Side navigation rail, typically for tablet/desktop layouts.

Syntax: `NavigationRail(destinations: [...], selectedIndex: i, onDestinationSelected: fn)`

Keyword Properties: `destinations`, `selectedIndex`, `onDestinationSelected`, `labelType`, `leading`, `trailing`, `extended`

Pseudo Code:

- place vertical rail in Row
- track selected destination
- update content pane

Example:

```
NavigationRail(  
  selectedIndex: idx,  
  onDestinationSelected: (i) => setState(() => idx = i),  
  destinations: const [  
    NavigationRailDestination(icon: Icon(Icons.inbox), label: Text('Inbox')),  
    NavigationRailDestination(icon: Icon(Icons.send), label: Text('Sent')),  
  ],  
)
```

8) FloatingActionButton

Description: Circular primary action button floating above content.

Syntax: FloatingActionButton(onPressed: fn, child: Icon(...))

Keyword Properties: onPressed, child, tooltip, backgroundColor, foregroundColor, heroTag

Pseudo Code:

- add floating button to Scaffold
- run primary action on press

Example:

```
FloatingActionButton(  
  onPressed: () {},  
  child: const Icon(Icons.add),  
)
```

9) BottomAppBar

Description: Material container at the bottom, often with FAB notch.

Syntax: BottomAppBar(child: Row(children: [...]))

Keyword Properties: child, color, elevation, shape, notchMargin, height

Pseudo Code:

- create bottom bar
- align actions inside row

Example:

```
BottomAppBar(  
  child: Row(  
    mainAxisAlignment: MainAxisAlignment.spaceAround,  
    children: const [Icon(Icons.home), Icon(Icons.person)],  
  ),  
)
```

10) Card

Description: Surface container with elevation and rounded corners.

Syntax: Card(child: Widget())

Keyword Properties: child, elevation, margin, color, shape, clipBehavior

Pseudo Code:

- wrap content in elevated surface
- style with margin and shape

Example:

```
Card(  
  margin: const EdgeInsets.all(12),  
  child: const Padding(  
    padding: EdgeInsets.all(16),  
    child: Text('Card content'),  
  ),  
)
```

11) ListTile

Description: Standard row layout for lists with leading, title, subtitle, trailing

Syntax: ListTile(leading: ..., title: ..., subtitle: ..., trailing: ...)
 Keyword Properties: leading, title, subtitle, trailing, onTap, selected, dense, contentPadding
 Pseudo Code:
 show row item
 optional icon and secondary text
 handle tap
 Example:
 ListTile(
 leading: const Icon(Icons.person),
 title: const Text('John Doe'),
 subtitle: const Text('Online'),
 trailing: const Icon(Icons.chevron_right),
 onTap: () {},
)

12) ListView
 Description: Scrollable linear list of widgets.
 Syntax: ListView(children: [...])
 Keyword Properties: children, builder, itemCount, padding, scrollDirection, physics, shrinkWrap
 Pseudo Code:
 create scrolling column
 render child items lazily with builder when needed
 Example:
 ListView.builder(
 itemCount: users.length,
 itemBuilder: (context, i) => ListTile(title: Text(users[i].name)),
)

13) GridView
 Description: Scrollable 2D layout for tiles or cards.
 Syntax: GridView.count(crossAxisCount: 2, children: [...])
 Keyword Properties: gridDelegate, children, builder, crossAxisSpacing, mainAxisSpacing, childAspectRatio
 Pseudo Code:
 define grid delegate
 build each tile
 allow vertical scrolling
 Example:
 GridView.count(
 crossAxisCount: 2,
 children: List.generate(6, (i) => Card(child: Center(child: Text('Item \$i')))),
),

14) SingleChildScrollView
 Description: Makes one child scrollable when content overflows.
 Syntax: SingleChildScrollView(child: Widget())
 Keyword Properties: child, scrollDirection, padding, reverse, physics
 Pseudo Code:
 wrap large form/page
 enable scroll for overflow
 Example:
 SingleChildScrollView(
 padding: const EdgeInsets.all(16),
 child: Column(children: const [Text('Long content')]),
)

15) Column
 Description: Vertical layout of children.
 Syntax: Column(children: [...])

Keyword Properties: children, mainAxisAlignment, crossAxisAlignment, mainAxisAlignment, spacing

Pseudo Code:

- stack children top to bottom
- control alignment and spacing

Example:

```
Column(  
  crossAxisAlignment: CrossAxisAlignment.start,  
  children: const [Text('Name'), Text('Email')],  
)
```

16) Row

Description: Horizontal layout of children.

Syntax: Row(children: [...])

Keyword Properties: children, mainAxisAlignment, crossAxisAlignment, mainAxisAlignment, spacing

Pseudo Code:

- place children left to right
- adjust alignment rules

Example:

```
Row(  
  mainAxisAlignment: MainAxisAlignment.spaceBetween,  
  children: const [Text('Price'), Text('\$99')],  
)
```

17) Stack

Description: Overlapping layout where children are painted in order.

Syntax: Stack(children: [...])

Keyword Properties: children, alignment, fit, clipBehavior

Pseudo Code:

- place background child
- overlay other children

Example:

```
Stack(  
  children: [  
    Image.network(imageUrl),  
    const Positioned(bottom: 8, right: 8, child: Icon(Icons.favorite)),  
  ],  
)
```

18) Positioned

Description: Precisely positions a child inside Stack.

Syntax: Positioned(top: 0, left: 0, child: Widget())

Keyword Properties: top, left, right, bottom, width, height, child

Pseudo Code:

- inside Stack
- anchor widget using offsets

Example:

```
const Positioned(  
  top: 12,  
  right: 12,  
  child: CircleAvatar(radius: 12),  
)
```

19) Container

Description: General-purpose box for size, padding, margin, decoration, and transform.

Syntax: Container(width: 100, padding: ..., decoration: ...)

Keyword Properties: width, height, padding, margin, alignment, color, decoration, child

Pseudo Code:

- create visual box

apply spacing and style
render child
Example:

```
Container(  
  padding: const EdgeInsets.all(12),  
  decoration: BoxDecoration(borderRadius: BorderRadius.circular(12), color: Colors.blue.shade50),  
  child: const Text('Badge'),  
)
```

20) SizedBox
Description: Fixed or constrained empty space or wrapper with explicit dimension S.
Syntax: `SizedBox(width: 16, height: 16, child: Widget())`
Keyword Properties: width, height, child
Pseudo Code:
 reserve space
 optionally constrain child
Example:

```
const SizedBox(height: 12)
```

21) Padding
Description: Insets around a child.
Syntax: `Padding(padding: EdgeInsets.all(8), child: Widget())`
Keyword Properties: padding, child
Pseudo Code:
 apply inner spacing around child
Example:

```
const Padding(  
  padding: EdgeInsets.symmetric(horizontal: 16),  
  child: Text('Padded text'),  
)
```

22) Center
Description: Centers a child within available space.
Syntax: `Center(child: Widget())`
Keyword Properties: child, widthFactor, heightFactor
Pseudo Code:
 occupy parent area
 align child to center
Example:

```
const Center(child: CircularProgressIndicator())
```

23) Align
Description: Aligns child to a custom point within parent.
Syntax: `Align(alignment: Alignment.topRight, child: Widget())`
Keyword Properties: alignment, widthFactor, heightFactor, child
Pseudo Code:
 set alignment point
 place child accordingly
Example:

```
const Align(  
  alignment: Alignment.bottomCenter,  
  child: Text('Footer'),  
)
```

24) Expanded
Description: Expands child to fill available space in Row/Column/Flex.
Syntax: `Expanded(flex: 1, child: Widget())`
Keyword Properties: flex, child
Pseudo Code:
 in Flex parent

divide remaining space by flex values

Example:

```
Row(children: const [Expanded(child: Text('A')), Expanded(child: Text('B'))])
```

25) Flexible

Description: Flexible sizing in Row/Column with loose or tight fit.

Syntax: Flexible(fit: FlexFit.loose, child: Widget())

Keyword Properties: flex, fit, child

Pseudo Code:

allow adaptive child sizing in flex layout

Example:

```
Row(children: const [Flexible(child: Text('Long text that can wrap'))])
```

26) Wrap

Description: Flow layout that wraps children into multiple lines.

Syntax: Wrap(spacing: 8, runSpacing: 8, children: [...])

Keyword Properties: children, spacing, runSpacing, alignment, runAlignment, direction

Pseudo Code:

place item chips in run

move to next line on overflow

Example:

```
Wrap(
  spacing: 8,
  children: const [Chip(label: Text('One')), Chip(label: Text('Two'))],
)
```

27) Text

Description: Displays styled text.

Syntax: Text('Hello', style: TextStyle(...))

Keyword Properties: style, textAlign, maxLines, overflow, softWrap

Pseudo Code:

render text with style

apply overflow rules

Example:

```
const Text(
  'Welcome',
  style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold),
)
```

28) RichText

Description: Multi-style text using TextSpan tree.

Syntax: RichText(text: TextSpan(children: [...]))

Keyword Properties: text, maxLines, overflow, textAlign

Pseudo Code:

create root TextSpan

add child spans with varying style

Example:

```
RichText(
  text: const TextSpan(
    style: TextStyle(color: Colors.black),
    children: [
      TextSpan(text: 'Hello '),
      TextSpan(text: 'Flutter', style: TextStyle(fontWeight: FontWeight.bold))
    ],
  ),
)
```

29) Icon

Description: Displays a graphical icon from Material icon font.

Syntax: Icon(Icons.home, size: 24)

Keyword Properties: icon, size, color, semanticLabel

Pseudo Code:

```
choose icon glyph
style size and color
```

Example:

```
const Icon(Icons.favorite, color: Colors.red)
```

30) Image

Description: Displays image from asset, network, memory, or file.

Syntax: Image.asset('assets/a.png')

Keyword Properties: image, fit, width, height, alignment, errorBuilder

Pseudo Code:

```
load image source
apply fit and bounds
```

Example:

```
Image.network(
  'https://picsum.photos/300',
  fit: BoxFit.cover,
  height: 160,
  width: double.infinity,
)
```

31) CircleAvatar

Description: Circular image/icon container, often used for user profile.

Syntax: CircleAvatar(radius: 20, backgroundImage: ...)

Keyword Properties: radius, backgroundColor, backgroundImage, foregroundImage, child

Pseudo Code:

```
render circular surface
place image or initials
```

Example:

```
const CircleAvatar(
  radius: 18,
  child: Text('TB'),
)
```

32) ElevatedButton

Description: Filled button with elevation for primary actions.

Syntax: ElevatedButton(onPressed: fn, child: Text('Save'))

Keyword Properties: onPressed, child, onPressed, style, autofocus

Pseudo Code:

```
create tappable button
execute handler on press
```

Example:

```
ElevatedButton(
  onPressed: submit,
  child: const Text('Submit'),
)
```

33) FilledButton

Description: Material 3 filled action button.

Syntax: FilledButton(onPressed: fn, child: Text('Continue'))

Keyword Properties: onPressed, child, style, onPressed

Pseudo Code:

```
render M3 filled control
run callback on click
```

Example:

```
FilledButton(
  onPressed: () {},
  child: const Text('Continue'),
)
```


34) OutlinedButton

Description: Button with outlined border and transparent background.

Syntax: OutlinedButton(onPressed: fn, child: Text('Cancel'))

Keyword Properties: onPressed, child, style, onLongPress

Pseudo Code:

- draw outlined control

- perform secondary action

Example:

```
OutlinedButton(  
  onPressed: () {},  
  child: const Text('Cancel'),  
)
```

35) TextButton

Description: Flat text-first button for low-emphasis actions.

Syntax: TextButton(onPressed: fn, child: Text('Skip'))

Keyword Properties: onPressed, child, style, onLongPress

Pseudo Code:

- display text action

- trigger callback

Example:

```
TextButton(  
  onPressed: () {},  
  child: const Text('Skip'),  
)
```

36) IconButton

Description: Icon-only button for compact actions.

Syntax: IconButton(icon: Icon(...), onPressed: fn)

Keyword Properties: icon, onPressed, tooltip, iconSize, color, style

Pseudo Code:

- show clickable icon

- handle tap callback

Example:

```
IconButton(  
  icon: const Icon(Icons.delete),  
  onPressed: () {},  
)
```

37) PopupMenuButton

Description: Button that shows a dropdown menu of actions.

Syntax: PopupMenuButton(itemBuilder: ..., onSelected: ...)

Keyword Properties: itemBuilder, onSelected, icon, tooltip, initialValue, offset

Pseudo Code:

- open menu on tap

- return selected value

- process action

Example:

```
PopupMenuButton<String>(  
  onSelected: (value) {},  
  itemBuilder: (context) => const [  
    PopupMenuItem(value: 'edit', child: Text('Edit')),  
    PopupMenuItem(value: 'delete', child: Text('Delete')),  
  ],  
)
```

38) DropdownButton

Description: Select one value from dropdown options.

Syntax: DropdownButton<T>(value: v, items: [...], onChanged: fn)

Keyword Properties: value, items, onChanged, hint, isExpanded, underline

Pseudo Code:

- show selected value

open menu for choices
set new value on selection

Example:

```
DropDownButton<String>(  
  value: selected,  
  onChanged: (v) => setState(() => selected = v!),  
  items: const [  
    DropdownMenuItem(value: 'A', child: Text('Option A')),  
    DropdownMenuItem(value: 'B', child: Text('Option B')),  
  ],  
)
```

39) TextField

Description: Basic text input widget.

Syntax: TextField(controller: c, decoration: InputDecoration(...))

Keyword Properties: controller, decoration, keyboardType, obscureText, onChanged, maxLines

Pseudo Code:

- render input box
- capture user typing
- validate or submit

Example:

```
TextField(  
  decoration: const InputDecoration(labelText: 'Email'),  
  onChanged: (v) {},  
)
```

40) TextFormField

Description: Form-integrated text field with validator support.

Syntax: TextFormField(validator: fn, onSave: fn)

Keyword Properties: controller, decoration, validator, onSave, autovalidateMode, keyboardType

Pseudo Code:

- place in Form
- run validator on submit
- save value if valid

Example:

```
TextFormField(  
  decoration: const InputDecoration(labelText: 'Password'),  
  obscureText: true,  
  validator: (v) => (v == null || v.length < 6) ? 'Min 6 chars' : null,  
)
```

41) Checkbox

Description: Boolean selection control.

Syntax: Checkbox(value: checked, onChanged: fn)

Keyword Properties: value, onChanged, activeColor, tristate, fillColor

Pseudo Code:

- bind bool state
- toggle on tap

Example:

```
Checkbox(  
  value: accept,  
  onChanged: (v) => setState(() => accept = v ?? false),  
)
```

42) Radio

Description: Single selection in a group.

Syntax: Radio<T>(value: x, groupValue: selected, onChanged: fn)

Keyword Properties: value, groupValue, onChanged, activeColor, toggleable

Pseudo Code:

- compare each value with selected group value

update group on change

Example:

```
Radio<String>(
  value: 'male',
  groupValue: gender,
  onChanged: (v) => setState(() => gender = v!),
)
```

43) Switch

Description: On/off toggle with sliding thumb.

Syntax: Switch(value: isOn, onChanged: fn)

Keyword Properties: value, onChanged, activeColor, thumbColor, materialTapTargetSize

Pseudo Code:

- show current boolean state
- flip on interaction

Example:

```
Switch(
  value: notificationsOn,
  onChanged: (v) => setState(() => notificationsOn = v),
)
```

44) Slider

Description: Single value selector in numeric range.

Syntax: Slider(value: x, min: 0, max: 100, onChanged: fn)

Keyword Properties: value, min, max, divisions, label, onChanged

Pseudo Code:

- drag thumb along track
- map position to numeric value

Example:

```
Slider(
  value: volume,
  min: 0,
  max: 100,
  onChanged: (v) => setState(() => volume = v),
)
```

45) RangeSlider

Description: Selects a min and max value range.

Syntax: RangeSlider(values: RangeValues(a,b), min: 0, max: 100, onChanged: fn)

Keyword Properties: values, min, max, divisions, labels, onChanged

Pseudo Code:

- drag two thumbs
- update lower and upper bounds

Example:

```
RangeSlider(
  values: priceRange,
  min: 0,
  max: 1000,
  onChanged: (v) => setState(() => priceRange = v),
)
```

46) Chip

Description: Small compact element for tags, attributes, or actions.

Syntax: Chip(label: Text('Tag'))

Keyword Properties: label, avatar, deleteIcon, onDelete, backgroundColor, shape

Pseudo Code:

- render compact pill
- optional avatar and delete action

Example:

```
Chip(
  label: const Text('Flutter'),
)
```

```
    onDelete: () {},  
  )
```

47) FilterChip

Description: Multi-select chip for filtering collections.

Syntax: FilterChip(label: Text('A'), selected: s, onSelect: fn)

Keyword Properties: label, selected, onSelect, avatar, selectedColor, showCheckmark

Pseudo Code:

```
  keep selected boolean per chip  
  toggle on user tap
```

Example:

```
  FilterChip(  
    label: const Text('In Stock'),  
    selected: inStock,  
    onSelect: (v) => setState(() => inStock = v),  
  )
```

48) ChoiceChip

Description: Single-select chip option.

Syntax: ChoiceChip(label: Text('A'), selected: s, onSelect: fn)

Keyword Properties: label, selected, onSelect, avatar, selectedColor

Pseudo Code:

```
  compare selected choice key  
  update selected key on tap
```

Example:

```
  ChoiceChip(  
    label: const Text('Small'),  
    selected: size == 'S',  
    onSelect: (_) => setState(() => size = 'S'),  
  )
```

49) InputChip

Description: Chip that can be selected, deleted, and tapped like contact chips.

Syntax: InputChip(label: Text('User'), onPressed: fn, onDelete: fn)

Keyword Properties: label, avatar, onPressed, onSelect, onDelete, selected

Pseudo Code:

```
  render contact-like chip  
  allow select and delete actions
```

Example:

```
  InputChip(  
    label: const Text('alice@example.com'),  
    onDelete: () {},  
  )
```

50) DataTable

Description: Tabular data display with rows and columns.

Syntax: DataTable(columns: [...], rows: [...])

Keyword Properties: columns, rows, sortColumnIndex, sortAscending, headingRowHeight, DataRowMinHeight

Pseudo Code:

```
  define table columns  
  map each data item to DataRow
```

Example:

```
  DataTable(  
    columns: const [  
      DataColumn(label: Text('Name')),  
      DataColumn(label: Text('Age')),  
    ],  
    rows: const [  
      DataRow(cells: [DataCell(Text('Ana')), DataCell(Text('27'))]),  
    ],  
  )
```

)

51) LinearProgressIndicator

Description: Horizontal progress bar for determinate/indeterminate progress.

Syntax: LinearProgressIndicator(value: 0.4)

Keyword Properties: value, color, backgroundColor, minHeight, borderRadius

Pseudo Code:

```
if known progress use value
else show ongoing animation
```

Example:

```
const LinearProgressIndicator(value: 0.65)
```

52) CircularProgressIndicator

Description: Circular loading indicator.

Syntax: CircularProgressIndicator(value: null)

Keyword Properties: value, strokeWidth, color, backgroundColor

Pseudo Code:

```
show spinner while loading
```

Example:

```
const CircularProgressIndicator()
```

53) SnackBar

Description: Temporary message at bottom with optional action.

Syntax: SnackBar(content: Text('Saved'))

Keyword Properties: content, action, duration, behavior, backgroundColor, shape

Pseudo Code:

```
build snackbar
show via ScaffoldMessenger
```

Example:

```
ScaffoldMessenger.of(context).showSnackBar(
  const SnackBar(content: Text('Saved successfully')),
)
```

54) AlertDialog

Description: Modal dialog with title, content, and actions.

Syntax: AlertDialog(title: Text('Title'), content: Widget(), actions: [...])

Keyword Properties: title, content, actions, icon, shape, scrollable

Pseudo Code:

```
open dialog
await user action
close with result
```

Example:

```
showDialog(
  context: context,
  builder: (_) => AlertDialog(
    title: const Text('Delete item?'),
    content: const Text('This action cannot be undone.'),
    actions: [
      TextButton(onPressed: () => Navigator.pop(context), child: const Text('Cancel')),
    ],
  ),
)
```

55) SimpleDialog

Description: Simpler dialog mostly for selecting one option.

Syntax: SimpleDialog(title: Text('Pick'), children: [...])

Keyword Properties: title, children, contentPadding, shape, elevation

Pseudo Code:

```
open dialog with option list
return selected value
```

Example:

```

showDialog(
  context: context,
  builder: (_) => SimpleDialog(
    title: const Text('Choose role'),
    children: [
      SimpleDialogOption(onPressed: () => Navigator.pop(context, 'Admin'), child: const Text('Admin')),
    ],
  ),
)

```

56) BottomSheet / showModalBottomSheet

Description: Sheet that rises from the bottom for supplementary actions or details.

Syntax: showModalBottomSheet(context: c, builder: (_) => Widget())

Keyword Properties: builder, isScrollControlled, enableDrag, backgroundColor, shape

Pseudo Code:

```

trigger bottom sheet
render action list/content
dismiss on tap or drag

```

Example:

```

showModalBottomSheet(
  context: context,
  builder: (_) => const Padding(
    padding: EdgeInsets.all(16),
    child: Text('Sheet content'),
  ),
)

```

57) TabBar

Description: Horizontal tabs for switching between views.

Syntax: TabBar(tabs: [Tab(text: 'A'), Tab(text: 'B')])

Keyword Properties: tabs, controller, isScrollable, indicatorColor, labelColor

Pseudo Code:

```

connect to TabController
render tab labels
update selected tab

```

Example:

```

const TabBar(
  tabs: [Tab(text: 'Posts'), Tab(text: 'Likes')],
)

```

58) TabBarView

Description: Page area controlled by TabBar and TabController.

Syntax: TabBarView(children: [...])

Keyword Properties: children, controller, physics, viewportFraction

Pseudo Code:

```

map each tab to page child
sync with controller index

```

Example:

```

const TabBarView(
  children: [Center(child: Text('Posts')), Center(child: Text('Likes'))],
)

```

59) DefaultTabController

Description: Provides TabController to descendant TabBar/TabBarView.

Syntax: DefaultTabController(length: 2, child: Scaffold(...))

Keyword Properties: length, initialIndex, animationDuration, child

Pseudo Code:

```

create tab controller scope
include TabBar and TabBarView below

```

Example:

```
DefaultTabController(  
  length: 2,  
  child: Scaffold(  
    appBar: AppBar(bottom: const TabBar(tabs: [Tab(text: 'A'), Tab(text: 'B')]  
)),  
    body: const TabBarView(children: [Text('A'), Text('B')]),  
  ),  
)
```

60) Stepper

Description: Multi-step workflow UI for forms and onboarding.

Syntax: Stepper(steps: [...], currentStep: i, onStepContinue: fn)

Keyword Properties: steps, currentStep, onStepContinue, onStepCancel, onStepTapped, type

Pseudo Code:

```
keep current step index  
validate step  
move next/previous
```

Example:

```
Stepper(  
  currentStep: step,  
  onStepContinue: () => setState(() => step++),  
  onStepCancel: () => setState(() => step = (step - 1).clamp(0, 10)),  
  steps: const [  
    Step(title: Text('Account'), content: Text('Create account details')),  
    Step(title: Text('Profile'), content: Text('Complete profile info')),  
  ],  
)
```

61) DatePicker (showDatePicker)

Description: Material dialog to pick calendar date.

Syntax: showDatePicker(context: c, firstDate: d1, lastDate: d2, initialDate: d0)

Keyword Properties: context, initialDate, firstDate, lastDate, helpText, selectableDayPredicate

Pseudo Code:

```
open date picker dialog  
await selected date  
update field/state
```

Example:

```
final picked = await showDatePicker(  
  context: context,  
  initialDate: DateTime.now(),  
  firstDate: DateTime(2000),  
  lastDate: DateTime(2100),  
)
```

62) TimePicker (showTimePicker)

Description: Material dialog to pick time.

Syntax: showTimePicker(context: c, initialTime: TimeOfDay.now())

Keyword Properties: context, initialTime, helpText, confirmText, cancelText, builder

Pseudo Code:

```
open time picker  
await selected time  
update UI value
```

Example:

```
final picked = await showTimePicker(  
  context: context,  
  initialTime: TimeOfDay.now(),  
)
```

=====

Quick Index: Additional Material UI Widgets to Explore

- AlertDialog
- ActionChip
- Badge
- Banner
- BottomSheet
- CardTheme (theme helper)
- CheckboxListTile
- ChipTheme (theme helper)
- DateRangePickerDialog
- Dialog
- Dialog.fullscreen
- Divider
- DrawerHeader
- DropdownMenu
- ExpansionPanelList
- ExpansionTile
- FilledButton.tonal
- Form
- IconTheme (theme helper)
- Ink
- InkResponse
- InkWell
- LinearProgressIndicator
- ListWheelScrollView (non-material but commonly used)
- Material
- MaterialBanner
- MenuAnchor
- MenuBar
- NavigationDrawer
- NavigationDrawerDestination
- NavigationIndicator
- NavigationToolbar
- PaginatedDataTable
- Placeholder
- PopupMenuDivider
- RadioListTile
- RefreshIndicator
- ReorderableListView
- SearchAnchor
- SearchBar
- SearchView
- SegmentedButton
- SelectableText
- SliderTheme (theme helper)
- SnackbarAction
- Step
- SwitchListTile
- Tab
- Tooltip

Best Practices

- Start with MaterialApp + ThemeData for consistent styling.
- Prefer composable layout widgets (Row, Column, Expanded, Padding) over deeply nested Container.
- Use Material 3 controls (NavigationBar, FilledButton, NavigationDrawer) for modern UI.
- Keep UI state local where possible; move shared state to state management only when needed.
- Use Form + TextFormField for validation-driven input.
- Add semantic labels for accessibility and test key flows with widget tests.

End of Notes